

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Будусова Татьяна Максимовна _____
(подпись) (фамилия, имя, отчество)

Группы ИПо-42

Руководитель практической подготовки от профильной организации

Директор
(должность)

(подпись) _____ Еговцев Станислав Владимирович
(фамилия, имя, отчество)

МП

«16» марта 2026 года

Руководитель практической подготовки от института

(подпись) _____ Берестов Дмитрий Борисович
(фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

СОДЕРЖАНИЕ

Введение.....	3
Раздел 1. Организационно-подготовительный этап прохождения практики на предприятии.....	5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии.....	5
Раздел 2. Ознакомление со структурой и характером деятельности подразделения.....	7
2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении.....	7
2.2. Ознакомление с должностными и функциональными обязанностями.	11
Раздел 3. Работа на рабочих местах или в подразделениях предприятия.....	14
Тема 3.1. Ознакомление с организацией информационного обеспечения подразделения	14
Тема 3.2. Выполнение производственных заданий.....	17
Заключение	40
Список используемых источников.....	41

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности в ходе практики являлись: разработка требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
7. Планировать и реализовывать собственное профессиональное и личностное развитие;
8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
10. Использовать информационные технологии в профессиональной деятельности
11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В ходе работы на предприятии было проведено ознакомление с организацией деятельности таксопарка STAX (ООО «Стакс36»), специализирующегося на пассажирских перевозках через агрегаторы такси, управлении автопарком и сопровождении инвестиционной модели приобретения транспортных средств. Были изучены основные принципы функционирования предприятия, структура управления и взаимодействие между подразделениями, а также порядок ведения документации и использования информационных технологий в бизнес-процессах.

Ознакомление включало изучение структуры управления, где стратегическое руководство осуществляется директором предприятия, подчиняющимся которому находятся два управляющих, курирующих различные функциональные направления. Было проведено наблюдение за взаимодействием подразделений, включая административно-управленческий блок, финансово-бухгалтерское направление, отдел по работе с водителями, техническое направление и информационно-техническое сопровождение. Отмечена значимость координации работы всех подразделений для обеспечения эффективного функционирования предприятия.

В процессе работы проводилось изучение функциональных обязанностей сотрудников и порядка выполнения основных операций. Особое внимание уделялось ведению учета информации о водителях, транспортных средствах, финансовых операциях и инвестициях, включая работу с электронными таблицами, базами данных и специализированными программными комплексами. Для обеспечения корректности данных

использовались процедуры регулярного обновления информации и резервного копирования.

Особое внимание уделялось соблюдению правил техники безопасности на предприятии и на рабочем месте. Основные меры включали:

- проверку исправности электрических приборов и оборудования перед началом работы;
- соблюдение безопасного расстояния от рабочих устройств и кабельных соединений;
- правильное подключение и использование персональных компьютеров, оргтехники и периферийных устройств;
- недопущение перегрузки электрических сетей и использование защитных устройств;
- поддержание порядка на рабочем месте для исключения травм и повреждений оборудования.

Кроме того, было изучено использование информационных систем и программного обеспечения, необходимых для выполнения повседневных задач, контроль доступа к данным и обеспечение информационной безопасности. Обеспечение корректной работы программного обеспечения включало регулярное резервное копирование данных, проверку работоспособности баз данных и своевременное обновление систем.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

Местом прохождения практики является таксопарк STAX (ООО «Стакс36»). Предприятие осуществляет деятельность в сфере пассажирских перевозок посредством взаимодействия с агрегаторами такси, а также занимается управлением автопарком и инвестиционной моделью финансирования транспортных средств.

Организационная структура предприятия построена по линейно-функциональному принципу (рис. 2.1.1). Управление осуществляется директором предприятия, который осуществляет стратегическое руководство и контроль за деятельностью компании. Под его руководством находятся два управляющих, каждый из которых курирует отдельные функциональные направления. Управляющие играют ключевую роль в оперативном управлении различными отделами и обеспечивают выполнение задач в соответствии с поставленными целями.

Основные функциональные направления под управлением двух управляющих включают:

- Маркетинговый управляющий, который отвечает за взаимодействие с внешними партнерами, рекламные и маркетинговые стратегии, а также за привлечение инвестиций.
- Операционный управляющий, который координирует работу внутренних подразделений, таких как техническое направление, отдел по работе с водителями и информационно-техническая поддержка.

Вся структура управления направлена на обеспечение эффективности работы каждого функционального блока, и каждый из управляющих взаимодействует с различными подразделениями для выполнения ключевых бизнес-задач.

Основные функциональные направления, подконтрольные каждому из управляющих:

- Административно-управленческий блок;
- Финансово-бухгалтерское направление;
- Отдел по работе с водителями;
- Техническое направление (контроль состояния автопарка);
- Информационно-техническое направление.

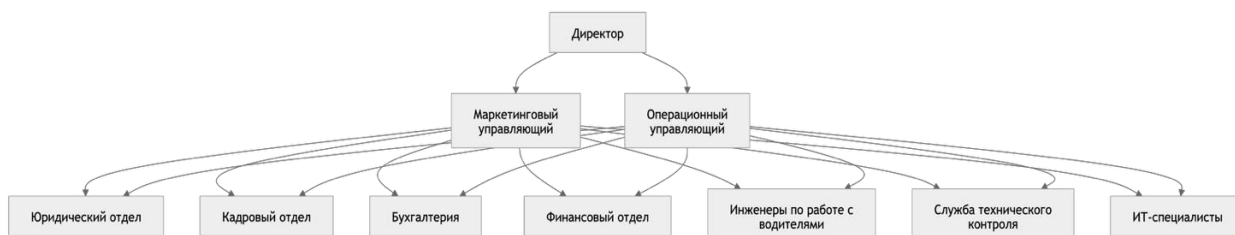


Рисунок 2.1.1 – организационная структура предприятия

ИТ-направление не выделено в крупное самостоятельное подразделение, однако играет важную роль в обеспечении функционирования всех бизнес-процессов предприятия.

Функционирование предприятия основано на тесном взаимодействии всех подразделений, так как деятельность таксопарка связана с постоянным движением финансовых, технических и информационных потоков.

Руководство предприятия осуществляет общее управление деятельностью организации, принимает стратегические решения по развитию автопарка, привлечению инвестиций, оптимизации расходов и внедрению информационных систем. Также руководство контролирует финансовые показатели и эффективность работы подразделений.

Финансово-бухгалтерское направление выполняет следующие функции:

- ведение бухгалтерского и налогового учета;
- расчет выплат водителям;
- расчет доходности по инвестициям;
- формирование финансовой отчетности;
- контроль поступления денежных средств от агрегаторов;

- анализ доходов и расходов предприятия.

Данное подразделение активно использует электронные таблицы, базы данных и специализированные учетные программы. Вся финансовая информация обрабатывается в цифровом формате, что требует надежной работы программного обеспечения.

Отдел по работе с водителями занимается:

- оформлением водителей и заключением договоров;
- контролем соблюдения условий сотрудничества;
- консультированием по вопросам эксплуатации автомобилей;
- взаимодействием с агрегаторами такси;
- урегулированием спорных ситуаций.

Информация о водителях хранится в электронных системах учета и регулярно обновляется.

Техническое направление отвечает за:

- контроль технического состояния автомобилей;
- планирование и организацию технического обслуживания;
- учет поломок и ремонтов;
- контроль затрат на автосервис;
- анализ повторяющихся неисправностей.

Данные о техническом состоянии автопарка также фиксируются в информационных системах предприятия.

Информационно-техническое направление обеспечивает цифровую поддержку всех процессов предприятия. В его задачи входят:

- сопровождение программного обеспечения;
- администрирование баз данных;
- настройка и поддержка рабочих станций;
- обеспечение резервного копирования данных;
- контроль корректности работы информационных систем;
- обеспечение защиты информации;
- разграничение прав доступа сотрудников к данным.

Взаимодействие между подразделениями осуществляется преимущественно в электронном формате. Данные о водителях, автомобилях, инвестициях и финансовых операциях являются взаимосвязанными и используются несколькими подразделениями одновременно. Это требует согласованной работы информационных систем и корректной организации баз данных.

Предприятие функционирует в режиме установленного рабочего графика с фиксированным временем начала и окончания рабочего дня. Работа сотрудников организована по кабинетному принципу с использованием персональных компьютеров.

Основная часть рабочих процессов автоматизирована. Документооборот ведётся преимущественно в электронном виде. Используются базы данных, электронные таблицы, специализированные учетные программы и онлайн-сервисы агрегаторов такси.

Форма организации труда предполагает распределение функциональных обязанностей между сотрудниками с использованием информационных технологий как основного инструмента работы.

Таксопарк STAX (ООО «Стакс36») является организацией, деятельность которой направлена на управление автопарком, организацию работы водителей и взаимодействие с агрегаторами такси. Особенностью предприятия является использование инвестиционной модели, при которой приобретение автомобилей может осуществляться за счет привлечения средств инвесторов.

Основными направлениями деятельности предприятия являются:

- Управление автопарком;
- Сопровождение договорных отношений с водителями;
- Ведение инвестиционного учета;
- Расчет и распределение доходов;
- Контроль технического состояния автомобилей;
- Анализ финансовых показателей.

Деятельность предприятия тесно связана с использованием информационных технологий. Все ключевые процессы — учет автомобилей, инвестиции, выплаты, договоры, отчётность — ведутся в электронном виде. Для хранения и обработки данных применяются базы данных и специализированные программные решения.

Высокая степень цифровизации процессов требует постоянного сопровождения программного обеспечения, своевременного обновления систем, резервного копирования данных и обеспечения информационной безопасности.

Предприятие ориентировано на повышение эффективности управления автопарком за счет автоматизации процессов и использования современных информационных технологий.

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

Деятельность специалиста ИТ-направления на предприятии направлена на обеспечение стабильной работы информационной инфраструктуры, сопровождение программного обеспечения и поддержку пользователей. Кроме того, специалист ИТ-направления активно взаимодействует с другими подразделениями компании, включая управление таксопарком, что позволяет эффективно интегрировать информационные технологии в операционные процессы.

Основные функции специалиста включают:

- Установку и настройку операционных систем;
- Установку, конфигурирование и обновление прикладного программного обеспечения;
- Администрирование баз данных;
- Настройку рабочих станций сотрудников;
- Контроль корректности работы информационных систем;
- Организацию резервного копирования данных;

- Обеспечение защиты информации;
- Техническую поддержку пользователей;
- Выполнение поручений от директора и управляющих, направленных на улучшение работы с информационными системами и процессами в рамках всего таксопарка;
- Контроль исправности всех систем, обеспечивающих работу таксопарка, включая серверное оборудование, сети и программные решения;
- Анализ и оптимизация процессов внутри таксопарка с точки зрения информационных технологий.

Специалист обеспечивает бесперебойное функционирование цифровых сервисов предприятия, а также принимает участие в модернизации программных решений и оптимизации рабочих процессов.

Специалист ИТ-направления обладает следующими правами:

- получение доступа к информационным ресурсам в рамках служебных обязанностей;
- запрос необходимой информации у сотрудников предприятия;
- внесение предложений по совершенствованию информационной инфраструктуры;
- участие в выборе программных и технических решений.

К обязанностям специалиста относятся:

- обеспечение работоспособности программного обеспечения;
- контроль актуальности версий программ и своевременное обновление;
- поддержание целостности и сохранности данных;
- настройка прав доступа пользователей;
- реализация мер по защите информации от несанкционированного доступа;
- выполнение резервного копирования и контроль хранения архивов;
- устранение программных и технических неисправностей.

Ответственность специалиста распространяется на корректность функционирования информационных систем и соблюдение требований информационной безопасности.

В рамках данной практики с руководителем от предприятия была поставлена следующая задача: провести диагностику всех компьютерных систем, установленных в таксопарке СТАХ, проверить базы данных, а также убедиться, что на всех рабочих станциях установлена антивирусная защита.

Задача включала в себя:

1. Проверку состояния всех компьютеров, установленных в офисе и на рабочих местах сотрудников, с акцентом на наличие актуального антивирусного ПО и безопасности системы.
2. Выполнение диагностики базы данных предприятия на предмет целостности данных, оптимизации запросов и производительности.
3. Проведение проверки и настройки системы резервного копирования, а также проверки восстановления данных из резервных копий.
4. Удостоверение в том, что все устройства на предприятии соответствуют корпоративным стандартам безопасности.
5. Улучшить структуры таблиц базы данных и добавить новые индексы для ускорения выполнения запросов и повышения общей производительности системы.
6. Модифицировать компоненты ПО для улучшения работы с данными и повышения производительности системы

По итогам выполнения задания должен быть подготовлен отчет для директора предприятия, в котором будет предоставлена информация о проделанных работах, выявленных проблемах, а также рекомендации по улучшению работы информационных систем и безопасности данных.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Ознакомление с организацией информационного обеспечения подразделения

Информационное обеспечение предприятия построено на использовании программных и технических средств, обеспечивающих автоматизацию ключевых бизнес-процессов. Деятельность таксопарка связана с обработкой значительного объема данных, включая сведения о водителях, автомобилях, инвестициях, финансовых операциях и договорных обязательствах. В связи с этим информационная система предприятия играет центральную роль в управлении организацией.

Таксопарк STAX использует собственную CRM систему, которая включает в себя все данные, связанные с работой таксопарка. В этой системе хранятся данные о сотрудниках, автомобилях, а также финансовая информация. Это позволяет управлять отношениями с клиентами и водителями, отслеживать финансовые потоки, включая расчеты с инвесторами и выплаты водителям. Система также обеспечивает удобное хранение и обработку информации о транспортных средствах, их техническом обслуживании и ремонте.

Кроме того, предприятие активно использует и сторонние системы для обеспечения эффективного функционирования бизнес-процессов:

1. ГЛОНАСС — система спутниковой навигации, используемая для мониторинга местоположения автомобилей таксопарка в реальном времени. С помощью ГЛОНАСС таксопарк может отслеживать маршруты автомобилей, оперативно изменять их маршруты для повышения эффективности логистики, а также обеспечивать безопасность водителей и пассажиров.

2. Скаут — специализированная система для управления автопарком. Включает в себя инструменты для контроля за техническим

состоянием автомобилей, ведения учета пробега, планирования технического обслуживания и ремонта транспортных средств. Также используется для анализа данных о расходах на топливо и состоянии автопарка в целом.

3. Microsoft Office — офисный пакет программных средств, широко используемый для подготовки документации, отчетности, бухгалтерских и юридических документов, а также для коммуникации с клиентами и партнерами. Включает в себя такие программы, как Word, Excel, PowerPoint и Outlook, которые активно используются в повседневной работе сотрудников.

4. Mail.ru — система для обмена электронной почтой и облачные сервисы, предоставляющие инструменты для хранения данных и совместной работы. Включает почтовый сервис, а также облачные хранилища для хранения отчетности, документов и других рабочих материалов, доступных для сотрудников таксопарка.

5. 1С:Бухгалтерия — система для ведения бухгалтерского и финансового учета. Используется для формирования отчетности, учета доходов и расходов, начисления выплат водителям и инвесторам, а также для налогового учета. Программа помогает эффективно управлять финансовыми потоками и соблюдать законодательные требования.

6. Яндекс.Такси (Яндекс Про) — приложение и платформа, используемая водителями и таксопарками для получения и выполнения заказов сервиса Яндекс.Такси. Система позволяет принимать заказы, отслеживать поездки, контролировать работу водителей и формировать данные о доходах.

Хранение и обработка структурированной информации организованы с использованием системы управления базами данных PostgreSQL. База данных содержит сведения об автопарке, инвестициях, договорах, выплатах, техническом обслуживании транспортных средств и иных показателях деятельности предприятия. Структура базы данных построена по принципам реляционной модели, что обеспечивает целостность, непротиворечивость и логическую связанность данных.

Процесс сбора информации осуществляется из нескольких источников. Данные о водителях и транспортных средствах вводятся сотрудниками предприятия при оформлении договоров и регистрации автомобилей. Финансовая информация поступает на основании отчетов агрегаторов такси и внутренних расчетов. Техническая информация фиксируется по результатам обслуживания и ремонта автомобилей. Все сведения регистрируются в электронных формах и заносятся в информационную систему.

Обработка информации включает проверку корректности введенных данных, их систематизацию, хранение и формирование отчетности. На основании накопленных данных формируются аналитические отчеты, расчеты выплат инвесторам и водителям, а также сводные финансовые показатели. Использование базы данных позволяет минимизировать вероятность ошибок, ускорить обработку информации и обеспечить оперативный доступ к необходимым сведениям.

Проектирование информационной системы предприятия основано на принципах структурированного хранения данных, разграничения прав доступа и обеспечения целостности информации. При эксплуатации системы применяется регулярное обновление программного обеспечения, контроль корректности функционирования баз данных и мониторинг производительности.

Особое внимание уделяется вопросам информационной безопасности. Поскольку предприятие осуществляет обработку персональных данных, реализованы меры по защите информации от несанкционированного доступа. Доступ к системе предоставляется на основе учетных записей с разграничением прав пользователей. Используются пароли с установленными требованиями к сложности.

Применяется антивирусная защита рабочих станций с регулярным обновлением сигнатурных баз. Организовано резервное копирование базы данных и критически важных файлов.

Тема 3.2. Выполнение производственных заданий

3.2.1. Использование выбранной системы контроля версий

В ходе прохождения практики на предприятии было проведено детальное ознакомление с системой контроля версий Git, которая является одной из ведущих технологий управления исходным кодом в современной разработке программного обеспечения. Git обеспечивает хранение полной истории изменений проекта, позволяет отслеживать внесенные корректировки и поддерживает работу нескольких разработчиков над одним и тем же проектом без потери данных.

Практическая работа включала следующие этапы взаимодействия с системой:

- Клонирование репозитория. Для начала работы использовался удалённый репозиторий, расположенный на платформе GitHub. С помощью команды `git clone` репозиторий копировался на локальный компьютер, что позволяло получить актуальную версию проекта для выполнения заданий на рабочем месте.
- Создание и редактирование файлов. После клонирования проекта выполнялось добавление новых файлов и внесение изменений в существующие. Все изменения фиксировались в локальном репозитории с помощью команды `git add`, что позволяло подготовить их для последующей фиксации в истории проекта.
- Фиксация изменений (коммиты). Каждое внесённое изменение оформлялось в виде коммита с подробным описанием проделанной работы. Это обеспечивало прозрачность и точное отслеживание хронологии изменений, а также позволяло при необходимости возвращаться к предыдущим состояниям файлов.
- Работа с ветками. Для организации параллельной разработки отдельных функциональных блоков создавались ветки. Создание ветки позволяло изолировать изменения и тестировать новые функции без влияния на основную версию проекта. В ходе работы выполнялось переключение

между ветками с помощью команды `git checkout`, что позволяло одновременно вести несколько потоков разработки.

- Слияние веток и разрешение конфликтов. После завершения работы над отдельной функциональностью выполнялось слияние веток с основной. В случае возникновения конфликтов происходил их анализ и разрешение, что позволяло объединить изменения без потери данных.

- Синхронизация с удалённым репозиторием. Завершённые изменения отправлялись на удалённый репозиторий с помощью команды `git push`, что обеспечивало актуальность общего проекта для всех участников и служило дополнительной резервной копией данных.

Использование Git способствовало развитию навыков планирования и организации работы над проектом. Система обеспечивала надежное хранение исходного кода, минимизацию риска потери информации и прозрачность всех изменений, что крайне важно при совместной работе сотрудников.

Особое внимание уделялось поддержанию правильной структуры репозитория, соблюдению стандартов оформления коммитов и ведению чистой истории изменений. Это позволяло анализировать выполненную работу, выявлять ошибки на ранних этапах и корректировать их без воздействия на функциональные элементы проекта.

3.2.2. Разработка программного обеспечения

В ходе прохождения практики было проведено комплексное выполнение задач по разработке программного обеспечения для управления базой данных партнёров и продукции. Работы включали выбор модели разработки, определение инструментов и среды программирования, построение проектной документации, создание и тестирование модулей системы, а также разработку интерфейса пользователя.

Для организации процесса разработки была выбрана итерационная модель разработки. Основным аргументом в пользу этого выбора являлась необходимость пошагового создания программного продукта с постепенной

проверкой функциональности на каждом этапе. Итерационная модель позволяла разделить проект на последовательные циклы: планирование, проектирование, разработка, тестирование и внедрение. Такой подход обеспечивал возможность выявления ошибок на раннем этапе, упрощал внесение изменений и корректировку функционала без риска нарушения структуры всей системы. Кроме того, итерационная модель оптимально подходила для практики в условиях ограниченного времени, поскольку позволяла получать рабочие результаты после каждой итерации, что способствовало последовательному освоению этапов проектирования и разработки.

В качестве инструментов для создания программного продукта использовались современные средства разработки: интегрированная среда Visual Studio с поддержкой языка C#, фреймворк .NET WPF для разработки графического интерфейса, а также система контроля версий Git, обеспечивающая управление изменениями и совместную работу над проектом. Для работы с базой данных использовалась система PostgreSQL, а взаимодействие с ней осуществлялось через Entity Framework Core, что обеспечивало автоматическую генерацию моделей данных и возможность безопасного взаимодействия с базой.

Среда программирования включала полный комплект инструментов для проектирования, написания и тестирования кода. Visual Studio предоставляла возможности визуального проектирования окон и элементов интерфейса, написания модульного кода, отладки и профилирования приложений. Использование WPF позволило создавать адаптивный и наглядный интерфейс, где блоки данных были представлены в виде панелей с фиксированными размерами, списками и таблицами (рис. 3.2.2.1).

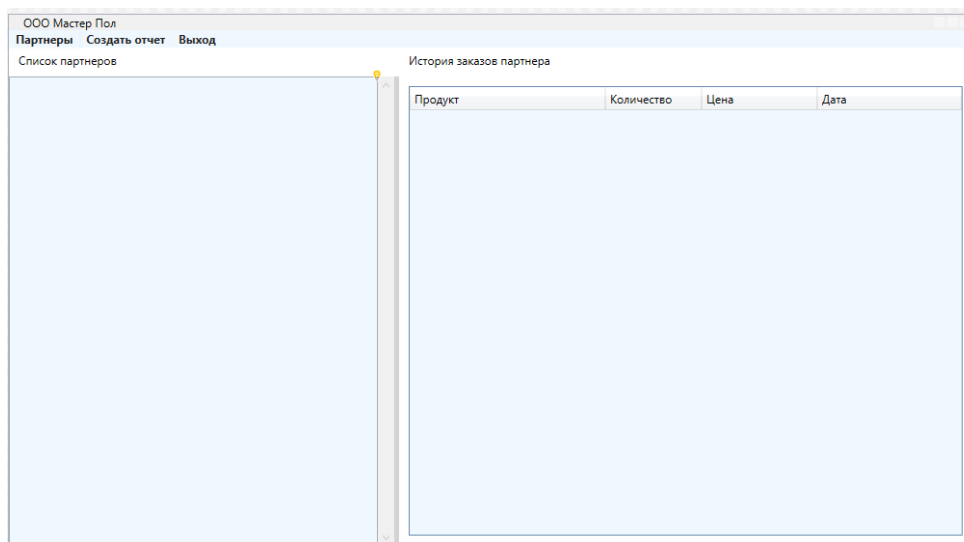


Рисунок 3.2.2.1 – Интерфейс главного окна программы

Разработка программного модуля производилась согласно установленным требованиям. Модуль включал функции добавления, редактирования и отображения данных, обработки продаж, вычисления суммарных показателей и скидок, а также работы с отчетностью. В процессе разработки соблюдались принципы модульности, читаемости и расширяемости кода, что позволило обеспечить удобство сопровождения и дальнейшего развития программного продукта.

Этапы разработки программного обеспечения включали следующие основные стадии:

- Выбор шаблона проекта с учетом структуры и целей системы;
- Составление схемы информационной базы данных с указанием всех таблиц, полей, связей и ограничений целостности;
- Документирование системы, включающее описание функций, структуры базы данных и логики работы модулей;
- Тестирование модулей и функциональных блоков, проверка корректности обработки данных и взаимодействия с интерфейсом;
- Сопровождение системы, включающее исправление ошибок, оптимизацию кода и поддержку актуальности базы данных.

Общий процесс разработки программного обеспечения представлен на рисунке 3.2.2.2.



Рисунок 3.2.2.2 – Этапы разработки программного обеспечения

На первом этапе разработки был выполнен выбор шаблона проекта в интегрированной среде разработки Microsoft Visual Studio 2022. В качестве основы был выбран шаблон WPF Application, предназначенный для создания настольных приложений с графическим пользовательским интерфейсом на платформе .NET.

Данный шаблон обеспечивает возможность разработки интерфейса с использованием языка разметки XAML, а также программной логики на языке C#. Использование технологии WPF позволяет разделить визуальную часть приложения и программную логику, что повышает удобство разработки и упрощает сопровождение системы.

После создания проекта была сформирована структура приложения, включающая:

- Главное окно приложения;
- Страница для добавления и редактирования данных;
- Классы бизнес-логики;

- Классы моделей данных;
- Подключение к базе данных PostgreSQL через Entity Framework Core.

Такой подход обеспечивает модульную архитектуру программного продукта и позволяет эффективно организовать взаимодействие между интерфейсом и базой данных.

Следующим этапом разработки являлось проектирование структуры информационной базы данных. Для хранения данных была выбрана система управления базами данных PostgreSQL, обладающая высокой надежностью и широкими возможностями для работы с реляционными структурами данных.

В процессе проектирования была разработана логическая структура базы данных, включающая таблицы:

- Типы партнеров;
- Партнеры;
- Типы продукции;
- Продукция;
- История реализации продукции партнерами.

Для каждой таблицы были определены:

- Первичные ключи;
- Внешние ключи;
- Ограничения целостности данных;
- Необходимые типы данных для хранения информации.

Связи между таблицами были реализованы по принципу «один-ко-многим», что соответствует требованиям нормализации базы данных и обеспечивает корректное хранение информации.

На основе разработанной структуры была построена ER-диаграмма базы данных, отражающая сущности системы и связи между ними (рис 3.2.2.3).

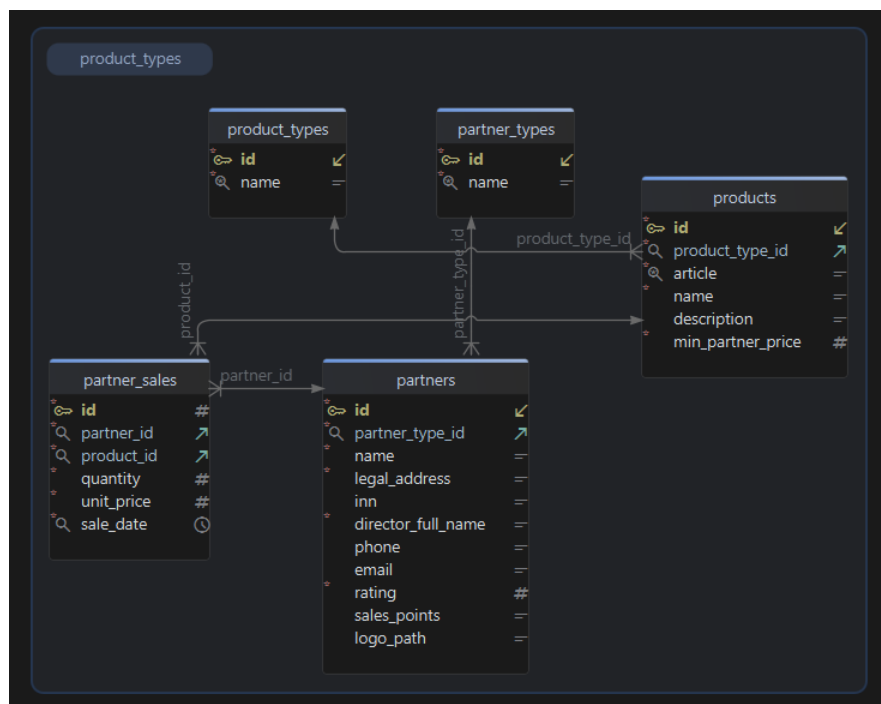


Рисунок 3.2.2.3 – ER-диаграмма базы данных

Использование данной схемы позволило наглядно представить структуру информационной системы и упростило дальнейшую реализацию программных модулей.

Важным этапом разработки являлось документирование программного продукта. Документация предназначена для описания структуры системы, принципов ее работы и особенностей реализации программных модулей.

В процессе разработки была подготовлена документация, включающая:

- Описание предметной области;
- Описание структуры базы данных;
- Ег-диаграмму базы данных;
- Диаграмму классов программного модуля;
- Описание алгоритмов работы системы;
- Описание пользовательского интерфейса.

Диаграмма классов отражает структуру программного модуля и взаимосвязи между основными объектами системы (рис. 3.2.2.4).

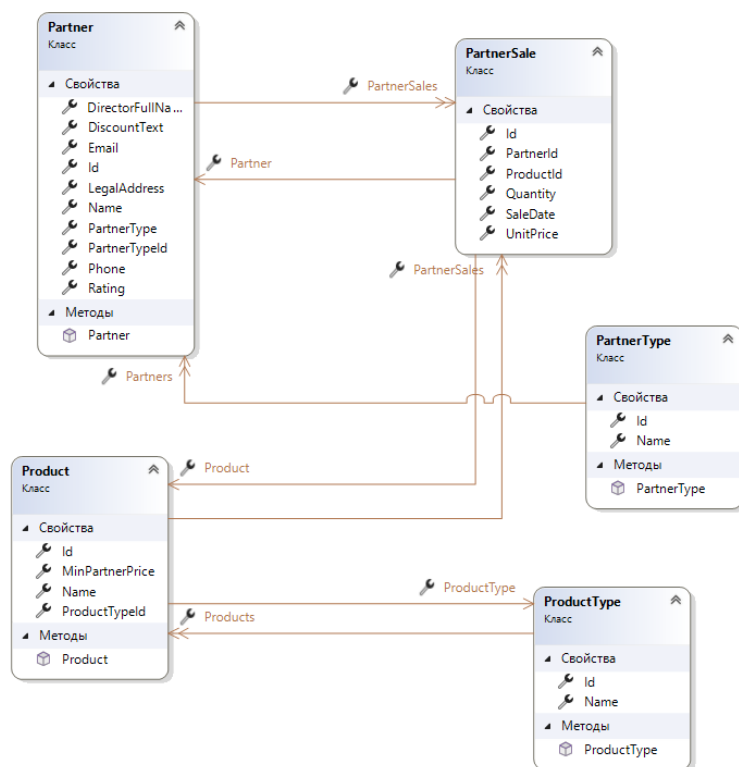


Рисунок 3.2.2.4 – Диаграмма классов

Основными классами системы являются:

- класс PartnerType, содержащий информацию о типах партнеров;
- класс Partner, содержащий сведения о партнерах компании;
- класс ProductType, содержащий типы продукции;
- класс Product, описывающий продукцию компании;
- класс PartnerSale, содержащий информацию о реализации

продукции партнерами.

Диаграмма классов позволяет структурировать программную архитектуру системы и обеспечивает наглядное представление связей между объектами.

После реализации основных модулей системы было выполнено тестирование программного обеспечения (рис. 3.2.2.5). Основной целью тестирования являлась проверка корректности работы приложения и выявление возможных ошибок.

В процессе тестирования проверялись следующие функции:

- Корректность подключения приложения к базе данных;

- Отображение списка партнеров;
- Добавление новых партнеров;
- Редактирование информации о партнерах;
- Сохранение изменений в базе данных;
- Отображение истории реализации продукции партнерами;
- Корректность расчета скидки партнера;
- Обработка некорректного ввода данных пользователем.

Особое внимание уделялось проверке обработки исключительных ситуаций. В случае возникновения ошибок система выводит информативные сообщения, позволяющие пользователю корректно выполнить необходимые действия. Подробное описание тестирования описано в пункте 3.2.6. «Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения» данного отчета.

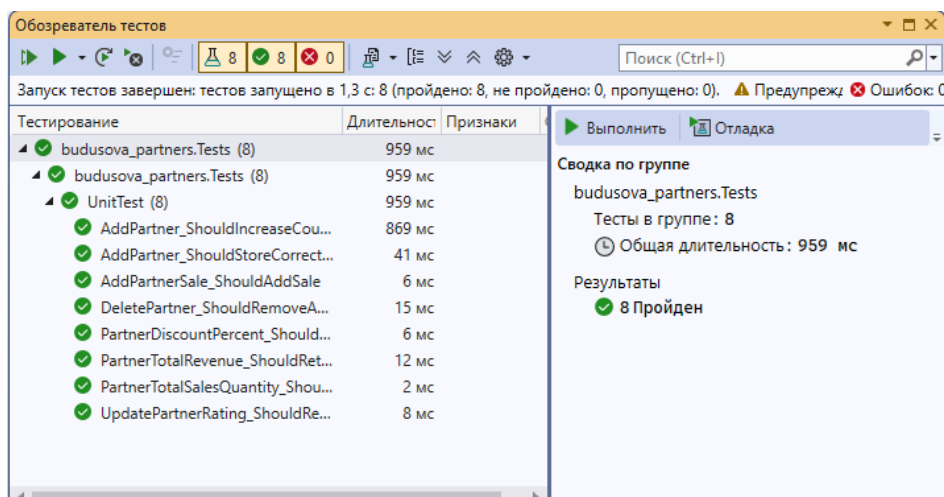


Рисунок 3.2.2.5 – Выполнение модульных тестов

Заключительным этапом разработки является сопровождение программного обеспечения. Сопровождение включает комплекс мероприятий, направленных на поддержание работоспособности системы и ее дальнейшее развитие.

Основными задачами сопровождения являются:

- Исправление обнаруженных ошибок;
- Оптимизация программного кода;
- Обновление структуры базы данных при необходимости;

- Улучшение пользовательского интерфейса;
- Расширение функциональности системы.

Для обеспечения удобства работы пользователей были разработаны формы для ввода, просмотра и редактирования данных. Формы позволяли удобно добавлять новых партнёров и продукцию, фиксировать продажи, просматривать историю транзакций и управлять типами объектов. Каждая форма предусматривала проверку корректности введённых данных, отображение сообщений об ошибках и визуальное оформление в соответствии с общим стилем системы (рис. 3.2.2.6).

Рисунок 3.2.2.6 – Интерфейс формы добавления\редактирования

Этапы разработки интерфейса включали:

- проектирование, в ходе которого определялось расположение элементов, логика взаимодействия с пользователем и структура окон;
- прототипирование, когда создавались макеты интерфейса, позволяющие визуально оценить расположение блоков, таблиц, кнопок и панелей данных;
- стилизация, обеспечивающая единый визуальный стиль, цветовое оформление, использование шрифтов и иконок, а также адаптацию интерфейса под разные разрешения экрана.

В результате выполненной работы был создан функциональный программный продукт, обеспечивающий автоматизацию учета партнёров и продукции, контроль продаж и формирование аналитических данных. Разработка обеспечила полное соответствие требованиям, повышала эффективность работы с данными и позволяла демонстрировать возможности системы на учебных и практических примерах (рис. 3.2.2.7).

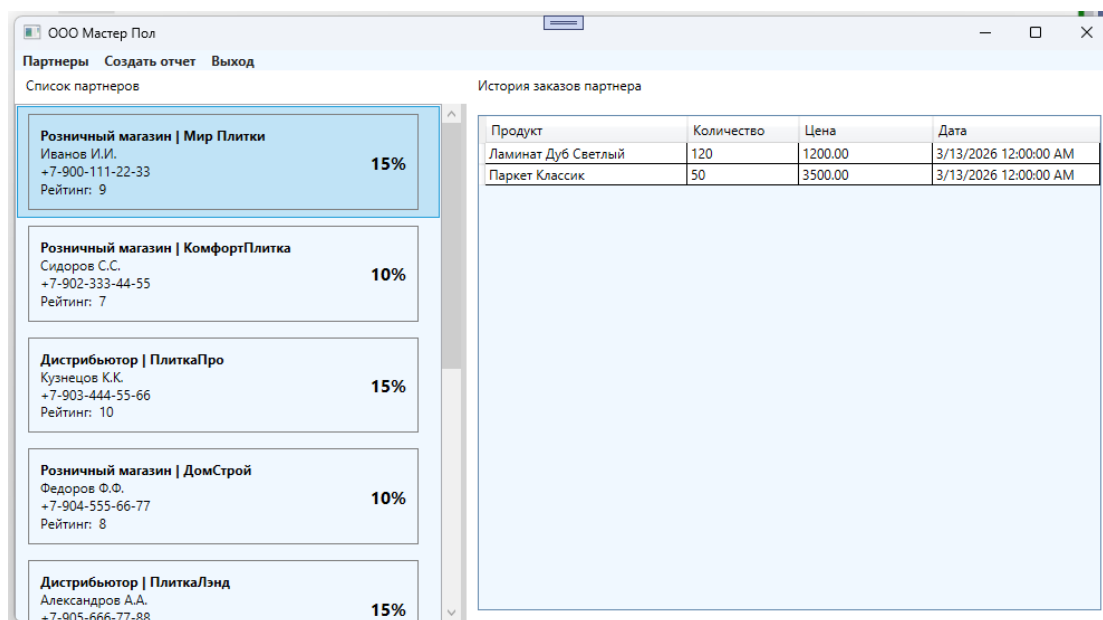


Рисунок 3.2.2.7 – Демонстрация работы программы

3.2.3. Использование методов для получения кода с заданной функциональностью и степенью качества

В процессе разработки программного обеспечения большое внимание уделялось обеспечению высокой функциональности кода и поддержанию его качества на всех этапах создания системы. Для достижения этих целей применялись комплексные методы, включающие структурное проектирование, модульное программирование, использование стандартов кодирования и внедрение процедур тестирования.

Для получения кода с необходимой функциональностью использовался подход модульного проектирования, при котором каждая логическая часть системы реализовывалась отдельным модулем. Каждый модуль имел определенные функции, чётко описанные входные и выходные данные, а также взаимодействовал с другими модулями через заранее определенные

интерфейсы. Такой подход позволял локализовать изменения, обеспечивать повторное использование кода и упрощал процесс сопровождения программного продукта.

Для гарантирования корректной работы и требуемой функциональности использовались следующие методы:

- Пошаговая проверка функциональных требований, при которой каждая разработанная функция тестировалась на соответствие установленным спецификациям;
- Применение шаблонов проектирования (Design Patterns), обеспечивающих стандартизированные решения для часто встречающихся задач, такие как управление данными, взаимодействие с базой и обработка событий интерфейса;
- Инкапсуляция и разделение ответственности внутри классов, что позволяло уменьшить количество ошибок при изменении кода и повысить читаемость системы;
- Использование средств автоматизации для генерации кода, таких как Entity Framework Core для работы с базой данных, что исключало ручное написание повторяющихся участков кода и снижало вероятность возникновения синтаксических и логических ошибок.

Для обеспечения высокой степени качества программного продукта применялись методы контроля и проверки кода, включающие:

- Модульное тестирование, когда каждая отдельная функция или метод проверялись на корректность выполнения задач и обработку возможных исключительных ситуаций;
- Интеграционное тестирование, направленное на проверку взаимодействия нескольких модулей системы, их совместимость и корректность обработки данных на уровне всего приложения;
- Ревью кода и использование статического анализа, позволяющие выявлять ошибки, нарушения стандартов кодирования, потенциальные проблемы производительности и уязвимости;

- Систематическое ведение документации кода, включая комментарии, описание функций и классов, что повышало удобство сопровождения и облегчало понимание логики программы другими разработчиками.

Кроме того, применялись методы контроля соответствия кода функциональным требованиям:

- Сравнение результатов выполнения функций с ожидаемыми данными;
- Проверка корректности обработки различных сценариев использования системы;
- Анализ производительности и устойчивости работы приложения при обработке большого объема данных, что было особенно важно при управлении базой партнёров, продукции и историей продаж.

Для поддержки качества интерфейса применялись принципы эргономики и адаптивного проектирования, что позволяло обеспечить удобство работы пользователя, минимизировать ошибки при вводе данных и повысить визуальную наглядность информации.

3.2.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

В ходе практики на предприятии было проведено детальное инспектирование компонентов разрабатываемого программного обеспечения с целью обеспечения их соответствия современным стандартам кодирования и требованиям качества. Инспекция кода является важной практикой, позволяющей выявлять ошибки на ранних этапах разработки, предотвращать возникновение дефектов в функциональности и повышать надёжность создаваемого продукта.

Использовались следующие виды инспекции:

- Построчная проверка кода (code review) – анализ отдельных файлов и методов на соответствие стилю кодирования, логике работы и

структурной организации. Данный метод позволяет выявлять ошибки логики, избыточные или дублирующиеся участки кода, а также недочёты в именовании переменных и функций.

- Функциональная инспекция – проверка соответствия реализованных компонентов заданным требованиям и функциональности, зафиксированной в техническом задании. Особое внимание уделялось корректности обработки данных и соблюдению алгоритмов, реализующих бизнес-логику.

- Автоматическая статическая инспекция – использование специализированных инструментов, анализирующих код на соответствие стандартам, выявляющих потенциальные ошибки и нарушающие практики.

Актуальными стандартами кодирования, которым следовали при разработке, являлись:

- Microsoft C# Coding Conventions – стандарты для языка программирования C#, включающие правила именования классов, методов, переменных, форматирования кода и отступов;

- SOLID-принципы – рекомендации по построению архитектуры и проектированию классов, обеспечивающие гибкость, расширяемость и поддержку кода;

- DRY (Don't Repeat Yourself) – принцип минимизации дублирования логики в коде для повышения читаемости и облегчения сопровождения.

Для проведения инспекции использовались современные программные средства, обеспечивающие автоматический и ручной анализ кода:

- Visual Studio Code / Visual Studio – встроенные инструменты проверки синтаксиса и статического анализа кода;

- ReSharper – расширение, предоставляющее автоматическую проверку соответствия кода стандартам, рекомендации по рефакторингу и обнаружение потенциальных ошибок;

- SonarLint – инструмент для анализа качества кода, выявляющий нарушения стандартов, уязвимости и проблемы производительности.

В процессе инспекции были проанализированы все ключевые компоненты разрабатываемого программного продукта, включая классы, модули работы с базой данных, интерфейсные элементы и алгоритмы расчёта бизнес-логики.

В ходе проверки фиксировались следующие результаты:

- устранение ошибок именования переменных и методов,
- исправление некорректной логики обработки данных,
- оптимизация повторяющихся участков кода,
- приведение форматирования к единому стилю.

3.2.5. Выполнение отладки программного модуля с использованием специализированных программных средств

В процессе разработки программного обеспечения выполнялась отладка созданного модуля с использованием специализированных средств разработки, что позволило выявить и устранить ошибки на ранних этапах. Процесс отладки был построен системно и включал несколько последовательных этапов.

Первоначально осуществлялась статическая проверка кода: проводился анализ исходных файлов на наличие синтаксических ошибок и несоответствий стандартам кодирования. Для этого использовались встроенные средства среды программирования Visual Studio 2022, включающие анализаторы кода и подсветку потенциально проблемных мест. Статическая проверка позволила обнаружить опечатки, неправильное использование типов данных, а также ошибки в именовании переменных и методов.

Далее переходили к динамической отладке с использованием встроенного отладчика Visual Studio. Основной подход заключался в следующем:

- Настройка точек останова (breakpoints) на ключевых участках кода, которые отвечают за обработку данных, взаимодействие с базой данных и выполнение бизнес-логики. Это позволяло останавливать выполнение программы в интересующих местах и проверять состояние переменных.
- Использование пошагового выполнения кода (Step Into, Step Over, Step Out) для анализа последовательности вызова методов и правильности работы алгоритмов. Такой подход позволял выявить ошибки логики и некорректное изменение состояния объектов.
- Контроль значений переменных и свойств объектов с помощью окна локальных переменных и Watch, что давало возможность проверять промежуточные результаты и соответствие данных ожидаемым значениям.
- Мониторинг работы с базой данных через SQL Profiler и средства логирования, чтобы убедиться, что запросы к таблицам формируются корректно, а данные сохраняются и извлекаются в ожидаемом формате.

Кроме того, для проверки работы отдельных компонентов использовались модульные тесты, которые выполняли автоматическую проверку корректности работы функций и методов. Это позволяло не только выявлять ошибки, но и контролировать стабильность кода при внесении изменений. Подробное описание создания и проведения модульного тестирования приведено в следующем пункте.

При выявлении ошибок фиксировались причины их возникновения, проводились корректировки исходного кода и повторное тестирование до получения корректного результата. Особое внимание уделялось проверке пограничных значений, обработке исключений и устойчивости системы к некорректным данным.

3.2.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

В ходе работы над программным обеспечением осуществлялась разработка тестовых наборов и тестовых сценариев, направленных на системную проверку функциональности и соответствия требований. Основная цель тестирования заключалась в выявлении ошибок на уровне модулей и обеспечения корректной работы программы при различных условиях эксплуатации (рис. 3.2.2.5).

Процесс тестирования включал следующие этапы:

- Анализ требований: На основе функциональных требований к системе определялись ключевые сценарии работы программы, основные классы и методы, которые необходимо проверить. Особое внимание уделялось критически важным операциям, связанным с хранением и обработкой данных о партнерах, продуктах и продажах.
- Формирование тестовых сценариев: Для каждого метода разрабатывались сценарии, включающие входные данные, ожидаемый результат и условия проверки. Сценарии охватывали как положительные варианты работы, так и проверку обработки некорректных или пограничных значений.
- Создание тестовых наборов: Сценарии объединялись в наборы тестов, которые позволяли последовательно проверять работу всей функциональности модуля. Каждый набор включал подготовку данных, выполнение операций и проверку результатов.
- Автоматизация тестирования через unit-тесты: Для реализации тестовых наборов использовались модульные тесты с применением `Microsoft.VisualStudio.TestTools.UnitTesting`. Для изоляции тестов и независимой проверки логики использовалась InMemory база данных, что позволяло безопасно проверять работу методов без воздействия на реальную базу данных.

Пример настройки тестовой среды:

```

var options = new
DbContextOptionsBuilder<AppDbContext>()
.UseInMemoryDatabase("TestDb")
.Options;
_context = new AppDbContext(options);

```

Для проверки корректности функциональности были созданы следующие unit-тесты:

Добавление нового партнера в базу данных. Тест проверял, что при создании нового объекта `Partner` общее количество записей увеличивается на единицу. Для этого вводились данные нового партнера (тип, наименование, ФИО директора, рейтинг) и проверялось, что база корректно сохраняет объект.

```

[TestMethod]
public void AddPartner_ShouldIncreaseCount()
{
    int before = _context.Partners.Count();
    _context.Partners.Add(new Partner
    {
        PartnerTypeId = 2,
        Name = "Test Partner 3",
        DirectorFullName = "Сидоров С.С.",
        Rating = 7
    });
    _context.SaveChanges();
    int after = _context.Partners.Count();
    Assert.AreEqual(before + 1, after);
}

```

Подсчет суммарного количества продаж партнера. Этот тест предназначался для проверки правильности вычисления общего количества проданных товаров конкретным партнером. В качестве входных данных использовались записи `PartnerSale`, а ожидаемым результатом была сумма значений `Quantity`.

```

[TestMethod]
public void

```

```

PartnerTotalSalesQuantity_ShouldReturnCorrectSum()
{
var totalQuantity = _context.PartnerSale
.Where(ps => ps.PartnerId == 1)
.Sum(ps => ps.Quantity);
Assert.AreEqual(70, totalQuantity);
}

```

Добавление и извлечение продукта. Тест проверял корректность сохранения объекта Product в базе данных и возможность его последующего извлечения по идентификатору. Вводились данные нового продукта (Id, тип продукта, наименование, минимальная цена), и проверялось, что извлечённый объект совпадает с введённым.

```

[TestMethod]
public void AddProduct_ShouldBeRetrievable()
{
var product = new Product
{
    Id = 3,
    ProductTypeId = 1,
    Name = "Product 3",
    MinPartnerPrice = 150
};
_context.Products.Add(product);
_context.SaveChanges();
var retrieved = _context.Products.Find(3);
Assert.IsNotNull(retrieved);
Assert.AreEqual("Product 3", retrieved.Name);
}

```

Проверка расчета общей стоимости продажи. Тест был направлен на подтверждение правильности вычисления итоговой стоимости продажи для конкретной записи PartnerSale. Использовались значения Quantity и UnitPrice, а результат сравнивался с ожидаемым.

```

[TestMethod]
public void

```

```

PartnerSaleTotalPrice_ShouldBeCalculatedCorrectly()
{
    var sale = _context.PartnerSale.First(ps => ps.Id == 1);
    decimal totalPrice = sale.Quantity * sale.UnitPrice;
    Assert.AreEqual(50 * 120, totalPrice);
}

```

Особое внимание уделялось проверке:

- правильности сохранения данных в базе;
- корректности вычислений (например, суммарного количества продаж и общей стоимости);
- устойчивости системы к некорректным или отсутствующим данным.

3.2.7. Интегрирование программных модулей

В процессе осуществлялось интегрирование отдельных программных модулей, разработанных для системы управления партнерами, продуктами и продажами, в единое функциональное приложение. Этот процесс был направлен на обеспечение корректного взаимодействия всех компонентов системы, согласованности данных и соблюдения требований к функциональности. Интеграция выполнялась после завершения разработки отдельных модулей и успешного прохождения их модульного тестирования, что позволяло убедиться в корректной работе каждого компонента в изоляции.

Процесс интеграции включал объединение модулей работы с базой данных, бизнес-логики и пользовательского интерфейса. Особое внимание уделялось взаимодействию модулей через заранее определенные интерфейсы и классы данных, что позволило избежать конфликтов при передаче параметров и обеспечило совместимость методов. Например, модуль управления партнерами взаимодействовал с модулем учета продаж посредством объектов `Partner` и `PartnerSale`, передавая идентификаторы и значения для вычисления рейтингов и скидок, а модуль управления

продуктами предоставлял информацию о минимальной цене и типах товаров для корректного формирования записей продаж.

Каждый этап интеграции сопровождался тестированием функциональности на уровне системы. Проверялась правильность выполнения операций: добавление и редактирование партнеров, учет и распределение продаж, расчет суммарных количеств и скидок, корректность отображения информации в интерфейсе. Для этих целей использовались как автоматические тесты с InMemory базой данных, так и ручное тестирование, позволяющее отслеживать состояние данных при последовательном выполнении бизнес-процессов.

Отладка интегрированных модулей осуществлялась с использованием встроенных средств среды разработки Visual Studio. Настройка точек останова, пошаговое выполнение кода и контроль состояния переменных позволяли выявлять ошибки на границах взаимодействия модулей и обеспечивать корректное формирование данных. В процессе выявлялись и устранялись ошибки передачи данных, несовпадения типов и некорректные вызовы методов, что обеспечивало стабильность работы всей системы.

Результатом проведенной интеграции стало создание единого работоспособного приложения, в котором взаимодействие модулей обеспечивало целостность данных и выполнение всех функций, предусмотренных проектной документацией. Интеграция позволила добиться согласованности процессов управления партнерами, продуктами и продажами, обеспечила корректность вычислений и надежность пользовательского интерфейса.

3.2.8. Верификация и аттестация программного обеспечения

Проверка соответствия разработанного программного обеспечения установленным требованиям осуществлялась систематически и включала комплекс мероприятий, направленных на подтверждение корректности работы всех модулей, их взаимодействия и соответствия функциональным,

техническим и качественным стандартам. Данный этап обеспечивал формальное подтверждение готовности продукта к эксплуатации и соответствие разработанного решения ожиданиям пользователей и регламентам предприятия.

Сначала выполнялась верификация требований, заключавшаяся в оценке того, насколько реализованный функционал соответствует техническому заданию и проектной документации. Проверялось, что все заявленные функции работают корректно, данные, вводимые пользователем, и результаты обработки соответствуют заданным форматам и ограничениям. Для этого использовались как ручные проверки, так и автоматизированные тестовые сценарии, позволяющие фиксировать ошибки на ранних этапах. Особое внимание уделялось проверке граничных и экстремальных значений, а также обработке некорректных вводных данных.

На следующем этапе осуществлялась верификация кода программного обеспечения, включавшая проверку соблюдения стандартов кодирования, правильности структурирования программных модулей и корректности использования типов данных. Производилась инспекция исходного кода, включающая проверку именования классов, методов и переменных в соответствии с соглашением CamelCase, анализ архитектурных зависимостей между модулями, а также соответствие принципам SOLID и рекомендациям по разработке на платформе C#. Инспекция проводилась вручную с использованием встроенных средств среды разработки Visual Studio, включая статический анализ кода, а также с привлечением средств автоматической проверки качества и покрытия модульными тестами.

Далее проводилась аттестация программного обеспечения, предполагающая комплексную проверку работы всех модулей в интегрированном виде. Аттестация включала тестирование взаимодействия модулей, проверку корректности обработки данных, соответствия расчетов заявленной бизнес-логике, а также устойчивости системы к некорректным и граничным значениям. Для аттестации разрабатывались интеграционные

тестовые сценарии, охватывающие критические функции, такие как добавление партнеров и продуктов, формирование отчетов, расчет суммарных показателей и применение скидок.

Особое внимание уделялось следующему:

- Контроль корректности взаимодействия модулей: проверка передачи данных между компонентами, синхронизации с базой данных, корректности расчетов на основе входных данных.
- Анализ устойчивости и обработки ошибок: тестирование сценариев с некорректным вводом, проверка генерации исключений и логирования событий.
- Проверка пользовательского интерфейса и логики бизнес-процессов: оценка корректного отображения данных, соответствия формы ввода и вывода требованиям, согласованности элементов интерфейса.
- Документирование результатов тестирования: фиксация найденных ошибок, внесение корректировок в код, повторное тестирование до достижения требуемого уровня качества.

Применение комплексной верификации и аттестации позволило убедиться в соответствии программного обеспечения заданным требованиям, подтвердить его надежность и готовность к эксплуатации, а также снизить риски возникновения ошибок в процессе дальнейшего использования системы.

ЗАКЛЮЧЕНИЕ

В ходе прохождения производственной практики с 09 по 17 марта 2026 г. были закреплены теоретические знания и получен практический опыт по специальности 09.02.07 «Информационные системы и программирование». Практика позволила освоить ключевые профессиональные компетенции и применить их в условиях, максимально приближенных к реальному производству.

В ходе работы были достигнуты следующие результаты.

1. Выполнена интеграция модулей в программное обеспечение, обеспечена корректность взаимодействия компонентов.
2. Проведена отладка программного модуля с использованием специализированных средств, включая анализ кода и пошаговое выполнение алгоритмов.
3. Разработаны тестовые наборы и сценарии, включая модульные тесты для проверки корректности работы функций и методов.
4. Выполнено инспектирование компонентов программного обеспечения на соответствие стандартам кодирования и требованиям стиля.
5. Выбраны оптимальные способы решения задач, осуществлён поиск и анализ информации, необходимой для работы с программным обеспечением.
6. Практически применены навыки работы в команде, коммуникации, использования современных информационных технологий и профессиональной документации.

Практика позволила получить навыки разработки, тестирования, отладки и сопровождения программного обеспечения, а также подготовила к самостоятельной профессиональной деятельности в сфере информационных систем и программирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Агальцов В.П. Базы данных. В 2-х кн. Кн. 2. Распределенные и удаленные базы данных: Учебник / В.П. Агальцов. М.: ИД ФОРУМ: НИЦ Инфра-М, 2013. 272 с. URL: <http://znanium.com/bookread.php?book=372740>;

2. Ахметгалиева В.Р. Базы данных: Microsoft Access 2013 [Электронный ресурс]: учебно-методическое пособие/ Ахметгалиева В.Р., Галяутдинова Л.Р.— Электрон. текстовые данные.— Москва: Российский государственный университет правосудия, 2017.— 95 с.— Режим доступа: <http://www.iprbookshop.ru/86345.html>.— ЭБС «IPRbooks»

3. Заботина, Н. Н. Проектирование информационных систем : учебное пособие /

Н. Н. Заботина. — Москва : ИНФРА-М, 2020. — 331 с. — (Высшее образование: Бакалавриат). - ISBN 978-5-16-004509-2. - Текст : электронный. - URL: <https://znanium.com/catalog/product/1036508>

4. Стасышин В.М. Разработка информационных систем и баз данных [Электронный ресурс]: учебное пособие для СПО/ Стасышин В.М.— Электрон. текстовые данные.— Саратов: Профобразование, 2020.— 100с.— Режим доступа: <http://www.iprbookshop.ru/87389.html> — ЭБС «IPRbooks»;

5. Шаньгин, В. Ф. Информационная безопасность компьютерных систем и сетей : учеб. пособие / В.Ф. Шаньгин. — Москва : ИД «ФОРУМ» : ИНФРА-М, 2019. — 416 с. — (Среднее профессиональное образование). - ISBN 978-5-8199-0754-2. - Текст : электронный. - URL: <https://znanium.com/catalog/product/1009605>

Дополнительные источники:

1. Башлы, П. Н. Информационная безопасность и защита информации [Электронный ресурс] : Учебник / П. Н. Башлы, А. В. Бабащ, Е. К. Баранова. М.: РИОР, 2013.222 с.

2. Кумскова И.А. Базы данных: учебник / И.А. Кумскова.-2-е изд., стер. М.: КНОРУС, 2012.488с.

3. Максимов Н. В. Компьютерные сети: Учебное пособие / Н.В. Максимов, И.И. Попов. - 3-е изд., испр. и доп. М.: Форум, 2008.448 с.;

Интернет-источники:

1. Руководство по программному пакету ERWin:[Электронный ресурс]. URL: <http://www.xserver.ru/computer/database/erwin/2/1.shtml>

2. Информатика. Инструментарий технологии программирования: [Электронный ресурс]. URL: http://www.klgtu.ru/students/literature/inf_asu/600.html

3. Базы данных в Delphi:[Электронный ресурс]. URL: <http://http://www.maksakov-sa.ru/BDDelphi/index.html>